P3769R1
Clarification of placement new deallocation

Published Proposal, 2026-03-27

Author:
Lauri Vasama

Audience:
CWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Current draft:
vasama.github.io/wg21/D3769.html

Current draft source:
github.com/vasama/wg21/blob/main/D3769.bs

Abstract

Clarifies wording for selection of deallocation functions in placement new expressions.

Table of Contents

§ 1. Introduction

[P3492R2] was forwarded to CWG in Hagenberg. In addition to its main language additions, it provided a drive-by fix for the poorly specified selection of deallocation functions in placement new expressions. The wording changes were split into this paper to make it easier to review the changes happening concurrently in this area:

1. The wording improvements now separated into this paper.
2. The language changes in [P3492R2] (Sized deallocation for placement new).
3. The language changes in [P2719R5] (Type-aware allocation and deallocation).

In addition it is proposed that the wording changes in this paper be applied as a defect report.

§ 2. Implementation divergence

There are two cases of implementation divergence around the use of deallocation functions in placement new expressions:

1. Whether function templates are considered valid candidates and argument deduction is performed. The current standard says no, however EDG alone conforms, and only in the case of global deallocation function templates.
2. The current wording specifies that if a deleted or inaccessible deallocation function is selected, the program is ill-formed. GCC and EDG are non-conforming in this case.

```
template<int>
struct alloc {};

void* operator new(size_t, alloc<0>& a);

template<int I>
void operator delete(void*, alloc<I>& a); // #1

void operator delete(void*, alloc<0>& a) = delete; // #2

int main() {
    alloc<0> al;
    new (al) thrower();
}
```

Current implementation behaviour:

	As shown	With #1 removed	With #2 removed
Clang	**Warning: ambiguity**	**Error: #2 is deleted**	**Uses #1**
GCC	**—**	**—**	**Uses #1**
MSVC	**Error: #2 is deleted**	**Error: #2 is deleted**	**Uses #1**
EDG	**—**	**—**	**—**[1]
P3769	**—**	**Error: #2 is deleted**	**Uses #1**

— No deallocation function is selected ~ the memory is leaked.

🟢 Behaviour is conforming.

🔴 Behaviour is non-conforming.

🟡 The standard is ambiguous.

[1] If the operators are class specific, EDG selects an unambiguous function template (i.e. with #1 removed) only if it is not deleted. If the selected function is deleted, no diagnostic is issued.

§ 3. Wording

§ 3.1. [expr.new]

Modify [expr.new] as follows:

> If the *new-expression* does not begin with a unary `::` operator and the allocated type is a class type `T` or an array thereof, a search is performed for the deallocation function's name in the scope of `T`. Otherwise, or if nothing is found, the deallocation function's name is looked up by searching for it in the global scope.

> ~~A declaration of a placement deallocation function matches the declaration of a placement allocation function if it has the same number of parameters and, after parameter transformations ([del.fct]), all parameter types except the first are identical. If the lookup finds a single matching deallocation function, that function will be called; otherwise, no deallocation function will be called. If the lookup finds a usual deallocation function and that function, considered as a placement deallocation function, would have been selected as a match for the allocation function, the program is ill-formed.~~ For a non-placement allocation function, the normal deallocation function lookup is used to find the matching deallocation function ([expr.delete]). For a placement allocation function, the selection process is described below. In any case, the matching deallocation function (if any) shall be non-deleted and accessible from the point where the *new-expression* appears.

> For a placement allocation function, the matching deallocation function is selected as follows:
>
> - Each candidate that is a function template is replaced by the function template specializations (if any) generated using template argument deduction ([temp.over], [temp.deduct.call]) with arguments as specified below.
> - Each candidate whose parameter-type-list is not identical to that of the allocation function, ignoring their respective first parameters, is removed from the set of candidates.
> - Each candidate whose associated constraints (if any) are not satisfied ([temp.constr.constr]) is removed from the set of candidates.
> - If exactly one function remains, that function is selected.
> - Otherwise, no deallocation function is selected.
>
> If a usual deallocation function is selected, the program is ill-formed.
>
> [Note: A deallocation function with an additional trailing parameter compared to the allocation function is never matched, even if a default argument is provided. —end note]

The following examples are entirely new, but are not highlighted in green in order to improve readability:

[Example:

```
struct A {};
struct T { T(); };

void* operator new(std::size_t s, A& al); // #1
```

```
template<int = 0>
void operator delete(void* p, A& al); // #2

A al;
T* p = new (al) T; // OK, uses #1 and #2.
```

—end example]

[Example:

```
template<int I>
struct A {};
struct T { T(); };

void* operator new(std::size_t s, A<0>& al); // #1

template<int I>
void operator delete(void* p, A<I>& al);
void operator delete(void* p, A<0>& al);

A<0> al;
T* p = new (al) T; // OK, uses #1. No deallocation function is selected (tv
```

—end example]

[Example:

```
template<int I>
struct A {};
struct T { T(); };

void* operator new(std::size_t s, A<0>& al); // #1

template<int I> requires (I > 0)
void operator delete(void* p, A<I>& al);
void operator delete(void* p, A<0>& al); // #2

A<0> al;
T* p = new (al) T; // OK, uses #1 and #2.
```

—end example]

§ References

§ Informative References

[P2719R5]

Louis Dionne, Oliver Hunt. *Type-aware allocation and deallocation functions*. 12 May 2025. URL: https://wg21.link/p2719r5

[P3492R2]

Lauri Vasama. *Sized deallocation for placement new*. 17 February 2025. URL: https://wg21.link/p3492r2